

Midterm-Crisis Sheet

▼ Resources

[requirement.txt](#)

[week4_exercises.ipynb](#)

[week5_exercises.ipynb](#)

[quiz.ipynb](#)

[Django Documentation](#)

[PostgreSQL Documentation](#)

▼ Getting Started

Note that all library/dependencies are listed in [requirement.txt](#), use this with `pip install -r requirement.txt`

1. Install Python virtual environment

```
pip install virtualenv
```

2. Create virtual environment

```
py -m venv myenv
```

3. Navigate to `'myenv'` then activate the virtual environment

```
myenv/  
├── Include  
├── Lib  
└── pyvenv.cfg
```

```
└─ Scripts
   └─ activate.bat
```

4. Installing dependencies

```
pip install django psycopg2
```

OR

```
pip install -r requirement.txt
```

5. Start Django's Project

```
django-admin startproject mysite
```

```
mysite/
├─ manage.py
└─ mysite
```

6. Start app

```
python manage.py startapp myapp
```

```
mysite/
├─ manage.py
├─ mysite
└─ myapp
```

7. Config `DATABASES` in `mysite/settings.py`

```
# Database
# https://docs.djangoproject.com/en/4.2/ref/settings/#databases

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "mysitedb",
        "USER": "postgres",
        "PASSWORD": "password",
        "HOST": "localhost",
        "PORT": "5432",
    }
}
```

```
}  
}
```

8. Config `INSTALLED_APPS` in `mysite/settings.py`

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
    "myapp",  
]
```

9. To migrate models to database

```
python manage.py makemigrations
```

Verify Migrations, Then migrate

```
python manage.py migrate
```

To Open Django Shell

```
python manage.py shell
```

10. Run server

```
python manage.py runserver
```

▼ IPYNB

Note that all library/dependencies are listed in requirement.txt, use this with pip install -r requirement.txt

1. Installing dependencies

```
pip install django-extensions jupyter notebook==6.5.7 ipython==8.25.0 jupyter_server==2.14.1 jupyterlab==4.2.2 jupyterlab_server==2.27.2
```

2. Create `notebooks` directories

```
mkdir notebooks
```

2. Config `INSTALLED_APP` in `mysite/settings.py`

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
    "django_extensions",  
    "myapp",  
]
```

3. Start Jupyter Notebook

```
python manage.py shell_plus --notebook
```

4. Change Notebook kernel to `Django Shell-Plus`

5. Allow Unsafe-Async in Notebook

```
import os  
os.environ["DJANGO_ALLOW_ASYNC_UNSAFE"] = "true"
```

▼ PSQL With CLI

1. Connect to the database as `postgres`

```
psql -U postgres
```

2. Create Database `mysitedb`

```
CREATE DATABASE mysitedb;
```

Run .sql files using following command

```
psql -U postgres -d mysitedb -q -f pathtosqlfile
```

▼ CRUD

Example Model

```
from datetime import date  
  
from django.db import models
```

```

class Blog(models.Model):
    name = models.CharField(max_length=100)
    tagline = models.TextField()

    def __str__(self):
        return self.name

class Author(models.Model):
    name = models.CharField(max_length=200)
    email = models.EmailField()

    def __str__(self):
        return self.name

class Entry(models.Model):
    blog = models.ForeignKey(Blog, on_delete=models.CASCADE)
    headline = models.CharField(max_length=255)
    body_text = models.TextField()
    pub_date = models.DateField()
    mod_date = models.DateField(default=date.today)
    authors = models.ManyToManyField(Author)
    number_of_comments = models.IntegerField(default=0)
    number_of_pingbacks = models.IntegerField(default=0)
    rating = models.IntegerField(default=5)

    def __str__(self):
        return self.headline

```

```

from blogs.models import Blog, Author, Entry

```

▼ Creating objects

.save()

```

b = Blog(name="Beatles Blog", tagline="All the latest Beatles news.")
b.save()

```

.create()

```

Blog.objects.create(name="Beatles Blog", tagline="All the latest Beatles news.")

```

▼ Retrieving objects

.all()

```
Entry.objects.all()
```

.filter()

```
Entry.objects.filter(pub_date__year=2010)
```

.exclude()

```
Entry.objects.exclude(pub_date__year=2010)
```

.get()

```
Entry.objects.get(id=1)
```

▼ Common Lookup Expression

Lookup	Description
<code>exact</code>	Exact match (<code>=</code>)
<code>icontains</code>	Case-insensitive exact match
<code>contains</code>	Field contains a value (case-sensitive)
<code>icontains</code>	Case-insensitive contains
<code>in</code>	Value is in a list
<code>gt</code>	Greater than (<code>></code>)
<code>gte</code>	Greater than or equal (<code>>=</code>)
<code>lt</code>	Less than (<code><</code>)
<code>lte</code>	Less than or equal (<code><=</code>)
<code>startswith</code>	Value starts with (case-sensitive)
<code>istartswith</code>	Case-insensitive starts with
<code>endswith</code>	Value ends with (case-sensitive)
<code>iendswith</code>	Case-insensitive ends with
<code>range</code>	Value is between two values
<code>date</code>	Match specific date (on a <code>DateTimeField</code>)
<code>year</code> / <code>month</code> / <code>day</code>	Extract and filter on that part of a date
<code>isnull</code>	Field is null or not
<code>regex</code>	Match a regex pattern
<code>iregex</code>	Case-insensitive regex match

▼ Saving changes to objects

.save()

```
b = Blog.objects.get(id=1)
b.name = "New name"
b.save()
```

▼ Deleting objects

.delete()

```
e = Entry.objects.get(id=1)
e.delete()
```

```
Entry.objects.filter(pub_date__year=2005).delete()
```

▼ Lookups span relationships

```
Blog.objects.filter(entry__authors__name="Lennon")
Blog.objects.filter(entry__authors__name__isnull=True)
```

▼ QuerySet Methods

Method	Description	Return Type	Example Usage
<code>.all()</code>	Returns all rows as model instances	<code>QuerySet[Model]</code>	<code>Entry.objects.all()</code>
<code>.filter()</code>	Filters rows based on conditions	<code>QuerySet[Model]</code>	<code>Entry.objects.filter(rating__gte=5)</code>
<code>.exclude()</code>	Excludes rows that match the condition	<code>QuerySet[Model]</code>	<code>Entry.objects.exclude(blog=1)</code>
<code>.order_by()</code>	Sorts the result by specified fields	<code>QuerySet[Model]</code>	<code>Entry.objects.order_by('pub_date')</code>
<code>.values()</code>	Returns a list of dictionaries with selected fields	<code>QuerySet[dict]</code>	<code>Entry.objects.values('headline', 'rating')</code>
<code>.distinct()</code>	Removes duplicates from results	<code>QuerySet</code>	<code>Entry.objects.values('blog').distinct()</code>
<code>.annotate()</code>	Adds computed fields (for aggregation, functions, etc.)	<code>QuerySet</code>	<code>Entry.objects.annotate(comment_count=Count('id'))</code>

Method	Description	Return Type	Example Usage
<code>.aggregate()</code>	Calculates and returns a single summary value	<code>dict</code>	<code>Entry.objects.aggregate(total=Count('id'))</code>
<code>.count()</code>	Returns the number of rows in the QuerySet	<code>int</code>	<code>Entry.objects.filter(blog=1).count()</code>
<code>.exists()</code>	Returns <code>True</code> if the QuerySet returns at least one row	<code>bool</code>	<code>Entry.objects.filter(blog=1).exists()</code>
<code>.first()</code>	Returns the first object (or <code>None</code>)	<code>Model or None</code>	<code>Entry.objects.first()</code>
<code>.last()</code>	Returns the last object (or <code>None</code>)	<code>Model or None</code>	<code>Entry.objects.last()</code>
<code>.get()</code>	Returns a single object that matches the filter	<code>Model</code>	<code>Entry.objects.get(id=1)</code>

▼ Common Expressions

```
from django.db.models import *
```

Expression	Description	Example
<code>F()</code>	Reference a model field (used in annotations, filters, updates)	<code>F('price') * 0.9</code>
<code>Q()</code>	Build complex <code>OR</code> , <code>AND</code> , <code>NOT</code> queries	<code>Q(age__gt=18)</code>
<code>Value()</code>	Use a literal value in an expression	<code>Value('Unknown')</code>

▼ F() Expression

Used to compare values between fields or perform arithmetic operations on fields.

```
from django.db.models import F
```

```
Entry.objects.filter(number_of_comments__gt=F("number_of_pingbacks"))
```

```
Entry.objects.filter(number_of_comments__gt=F("number_of_pingbacks") * 2)
Entry.objects.filter(rating__lt=F("number_of_comments") + F("number_of_pingbacks"))
```

▼ Q() Object

Complex Lookup


```
from django.db.models import Q
```

```
Entry.objects.filter(Q(headline__startswith="Who") | Q(headline__startswith="What"))
```

```
Entry.objects.filter(Q(headline__startswith="Who") | ~Q(pub_date__year=2005))
```

```
Poll.objects.get(
    Q(question__startswith="Who"),
    (Q(pub_date=date(2005, 5, 2)) | Q(pub_date=date(2005, 5, 6))),
)
```

▼ Value()

```
from django.db.models import Value
```

```
Author.objects.annotate(
    screen_name=Concat("name", Value(" "), "goes_by", Value("")), output_field=CharField())
)
```

```
blogs = Blog.objects.annotate(
    label=Value('Main Blog', output_field=models.CharField())
)
```

▼ Common Database Functions

```
from django.db.models.functions import *
```

Function	Description	Example
<code>Lower()</code>	Converts a field to lowercase	<code>Lower('name')</code>
<code>Upper()</code>	Converts a field to uppercase	<code>Upper('name')</code>
<code>Length()</code>	Returns the length of a field (string)	<code>Length('name')</code>
<code>Concat()</code>	Combines fields and/or values	<code>Concat(F('first'), F('last'))</code>
<code>Greatest()</code>	Returns the largest value among two or more fields	<code>annotate(max_val=Greatest('field1', 'field2'))</code>
<code>Coalesce()</code>	Uses first non-null value from fields	<code>Coalesce('nickname', 'N/A')</code>
<code>Now()</code>	Gets current timestamp	<code>filter(created_at__lte=Now())</code>
<code>Extract()</code>	Extracts date/time part from a date field	<code>Extract('date', 'year')</code>

Function	Description	Example
<code>Trunc()</code>	Truncates a date field to a time unit	<code>Trunc('date', 'month')</code>

▼ Lower()

```
from django.db.models.functions import Lower
```

```
authors = Author.objects.annotate(
    lowercase_email=Lower('email')
)
```

▼ Upper()

```
from django.db.models.functions import Upper
```

```
blogs = Blog.objects.annotate(
    upper_name=Upper('name')
)
```

▼ Length()

```
from django.db.models.functions import Length
```

```
entries = Entry.objects.annotate(
    headline_length=Length('headline')
)
```

▼ Concat()

```
from django.db.models.functions import Concat
```

```
Author.objects.annotate(
    contact_info=Concat('name', Value(' <'), 'email', Value(' >'), output_field=CharField())
)
```

▼ Greatest()

```
from django.db.models.functions import Greatest
```

```
entries = Entry.objects.annotate(  
    max_feedback=Greatest('number_of_comments', 'number_of_pingbacks')  
)
```

▼ Coalesce()

```
from django.db.models.functions import Coalesce
```

```
Entry.objects.annotate(  
    rating_or_default=Coalesce('rating', Value(0))  
)
```

▼ Now()

```
from django.db.models.functions import Now
```

```
Entry.objects.filter(pub_date__lte=Now())
```

▼ Extract()

```
from django.db.models.functions import Extract
```

```
Entry.objects.annotate(  
    pub_year=Extract('pub_date', 'year')  
)
```

▼ Trunc()

```
from django.db.models.functions import Trunc
```

```
# Count how many entries were published in each month  
entries_per_month = Entry.objects.annotate(  
    pub_month=TruncMonth('pub_date')
```

```
).values('pub_month').annotate(  
    total=Count('id')  
)order_by('pub_month')
```

▼ Annotation

```
from datetime import date  
  
from django.db import models  
  
class Company(models.Model):  
    name = models.CharField(max_length=100)  
    ticker = models.CharField(max_length=20, null=True)  
    num_employees = models.IntegerField()  
    num_tables = models.IntegerField()  
    num_chairs = models.IntegerField()  
  
    def __str__(self):  
        return self.name
```

```
# How many chairs are needed for each company to seat all employees?  
>>> company = (  
    Company.objects.filter(num_employees__gt=F("num_chairs"))  
    .annotate(chairs_needed=F("num_employees") - F("num_chairs"))  
    .first()  
)  
>>> company.num_employees  
50  
>>> company.num_chairs  
30  
>>> company.chairs_needed  
20
```

Compared To SQL

```
SELECT *,  
    (num_employees - num_chairs) AS chairs_needed  
FROM company  
WHERE num_employees > num_chairs  
LIMIT 1;
```

▼ .values()

Returns a list of dictionaries with only the selected fields. Used for projections and enabling grouping when combined with `.annotate()`.

Select specific field from `QuerySet` Return `QuerySet[dict]`

```
Entry.objects.values('headline', 'rating')
```

▼ Grouping

```
Author.objects.values("name").annotate(average_rating=Avg("book__rating"))
```

▼ Aggregation

```
from django.db.models import *
```

Function	Use in Django	Description
<code>Count()</code>	<code>annotate(num_posts=Count('posts'))</code>	Count related objects or rows
<code>Sum()</code>	<code>annotate(total=Sum('price'))</code>	Total sum of a field
<code>Avg()</code>	<code>aggregate(average=Avg('score'))</code>	Average value
<code>Max()</code>	<code>annotate(latest=Max('created_at'))</code>	Maximum value
<code>Min()</code>	<code>aggregate(earliest=Min('joined'))</code>	Minimum value

▼ Count()

```
from django.db.models import Count
```

```
Publisher.objects.annotate(num_books=Count("book"))
```

```
Publisher.objects.annotate(above_4=Count("book", filter=Q(book__rating__gt=4)))
```

▼ Sum(), Avg(), Max(), Min()

```
from django.db.models import Sum, Avg, Max, Min
```

```
Book.objects.all().aggregate(Avg("price"))
```

```
Book.objects.aggregate(Sum("price"), Avg("price"), Max("price"), Min("price"))
```

▼ Subquery

```
class Blog(models.Model):
    body = models.TextField()
    modified = models.DateTimeField(auto_now=True)
```

```
class Comment(models.Model):
    body = models.TextField()
    length = models.IntegerField(default=0)
    email = models.EmailField(null=True)
    modified = models.DateTimeField(auto_now=True)
    created_at = models.DateTimeField(auto_now_add=True)
    blog = models.ForeignKey(Blog, on_delete=models.CASCADE)
```

▼ **Problem: What is latest comment in each Blog .**

1. Determine on how to find latest comment.

```
from django.db.models import OuterRef, Subquery
```

```
latest_comment = Comment.objects.filter(blog=OuterRef('pk')).order_by('-created_at')
```

2. What should be done to that comment.

```
latest_comment_text = latest_comment.values('body')[:1]
```

3. Use Subquery .

```
Blog.objects.annotate(latest_comment=Subquery(latest_comment_text))
```

SQL Equivalent

```
SELECT b.*, (
    SELECT c.body
    FROM comment c
    WHERE c.blog_id = b.id
    ORDER BY c.modified DESC
    LIMIT 1
) AS latest_comment_text
FROM blog b;
```

▼ **Problem: What is Sum of length of all comment in each Blog .**

```
from django.db.models import OuterRef, Subquery, Sum
comments = Comment.objects.filter(blog=OuterRef("pk")).order_by().values("blog")
total_comments = comments.annotate(total=Sum("length")).values("total")
Post.objects.annotate(comment_length=Subquery(total_comments))
```

▼ Many-To-One

```
class AAA(models.Model):
    a_foo1 = models.CharField(max_length=100)

class BBB(models.Model):
    b_foo1 = models.CharField(max_length=100)
    aaa = models.ForeignKey(AAA, on_delete=models.CASCADE)
```

Find **BBB** objects that related to **aaa** .

```
aaa = AAA.objects.first()
aaa.bbb_set.all()
```

Filter Example

```
aaa = AAA.objects.filter(bbb__b_foo1__in=('foo1', 'foo2'))
```

▼ Many-To-Many

```
class AAA(models.Model):
    A_foo1 = models.CharField(max_length=100)

class BBB(models.Model):
    B_foo1 = models.CharField(max_length=100)
    aaa = models.ManyToManyField(AAA)
```

Find all **BBB** objects that are related to a given **AAA** object.

```
target_aaa = AAA.objects.get(id=1)
```

```
BBB.objects.filter(aaa=target_aaa)
BBB.objects.filter(aaa__a_foo1='some value')
```

```
target_aaa.bbb_set.all()
```

Adding many-to-many to field

```
bbb = BBB.objects.get(id=1)
bbb.aaa.add(target_aaa)
```

```
bbb = BBB.objects.get(id=1)
```

```
target_aaa.bbb_set.add(bbb)
```

▼ Datetime

Python Datetime Module

Source

Datetime module มีทั้งหมด 5 class หลัก

1. `date` – มี attributes ได้แก่ year, month, และ day
2. `time` – มี attributes ได้แก่ hour, minute, second, microsecond, และ tzinfo
3. `datetime` – คือการรวม `date` และ `time` และมี attributes ได้แก่ year, month, day, hour, minute, second, microsecond, and tzinfo
4. `timedelta` – เป็นระยะเวลา (microsecond) ซึ่งเป็นส่วนต่างของ 2 date, time หรือ datetime
5. `tzinfo` – เป็น object สำหรับเก็บข้อมูล time zone

time

`time` class ใช้สำหรับเก็บ **ข้อมูลเวลา (ไม่รวมวันที่)** โดยจะประกอบด้วย attributes:

- `hour`
- `minute`
- `second`
- `microsecond`
- `tzinfo` (optional, time zone info)

ตัวอย่างการสร้าง object `time`

```
>>> from datetime import time

# กำหนดเวลา 12:34:56
>>> t = time(12, 34, 56)
>>> print(t)
12:34:56

# เพิ่ม microsecond
>>> t2 = time(12, 34, 56, 123456)
>>> print(t2)
12:34:56.123456

# เข้าถึง attributes
>>> print("Hour:", t.hour)
Hour: 12
>>> print("Minute:", t.minute)
```



```
Minute: 34
>>> print("Second:", t.second)
Second: 56
```

สร้างจาก `datetime.now().time()`

```
>>> from datetime import datetime

>>> now_time = datetime.now().time()
>>> print(now_time)
20:41:23.572404 # จะมี microsecond ติดมาด้วย

# ถ้าอยากได้แค่ HH:MM:SS
>>> print(now_time.strftime("%H:%M:%S"))
20:41:23

# หรือจะลบ microsecond ออก
>>> print(now_time.replace(microsecond=0))
20:41:23
```

datetime

Argument `year`, `month`, และ `day` จะต้องกำหนด (mandatory) และ `tzinfo` เป็น None ได้

Range ของ attribute แต่ละตัว:

- `MINYEAR <= year <= MAXYEAR`
- `1 <= month <= 12`
- `1 <= day <= number of days in the given month and year`
- `0 <= hour < 24`
- `0 <= minute < 60`
- `0 <= second < 60`
- `0 <= microsecond < 1000000`

```
>>> a = datetime(1999, 12, 12, 12, 12, 12, 342380)
>>> print(a)
1999-12-12 12:12:12.342380
```

timedelta

```
>>> from datetime import datetime, timedelta
```

```

# Using current time
>>> ini_time_for_now = datetime.now()

# printing initial_date
>>> print("initial_date", str(ini_time_for_now))
initial_date 2024-07-13 23:45:16.572404

# Calculating future dates
# for two years
>>> future_date_after_2yrs = ini_time_for_now + timedelta(days=730)

>>> future_date_after_2days = ini_time_for_now + timedelta(days=2)

# printing calculated future_dates
>>> print('future_date_after_2yrs:', str(future_date_after_2yrs))
future_date_after_2yrs: 2026-07-13 23:45:16.572404
>>> print('future_date_after_2days:', str(future_date_after_2days))
future_date_after_2days: 2024-07-15 23:45:16.572404

```

Python datetime.tzinfo()

The `datetime.now()` function จะไม่มีการเก็บข้อมูล time zones

การเก็บข้อมูล time zones จะใช้ `tzinfo` ซึ่งเป็น abstract base class ใน Python

เราสามารถส่ง instance ของ class `tzinfo` ใน constructors ของ object datetime and time เพื่อใช้ในการกำหนด time zones

Naive and Aware datetime objects

- **Naive datetime objects** หมายถึง datetime object ที่ไม่มีการกำหนดข้อมูล time zone (tzinfo เป็น None)
- **Aware datetime objects** คือ datetime object ที่มีข้อมูล time zone

```

>>> from zoneinfo import ZoneInfo
>>> from datetime import datetime

>>> dt1 = datetime(2015, 5, 21, 12, 0)
>>> print(dt1)
2015-05-21 12:00:00
>>> dt2 = datetime(2015, 12, 21, 12, 0, tzinfo = ZoneInfo(key='Asia/Bangkok'))
>>> print(dt2)
2015-12-21 12:00:00+07:00

>>> print("Naive Object :", dt1.tzname())

```

```
Naive Object : None
>>> print("Aware Object :", dt2.tzname())
Aware Object : +07

>>> now_aware = dt1.replace(tzinfo=ZoneInfo(key='UTC'))
>>> print(now_aware)
2015-05-21 12:00:00+00:00
```

Django - Time zones

Source

ถ้ามีการ enable time zone support (USE_TZ=True) โดย default Django จะบันทึก datetime information ในฐานข้อมูลเป็น UTC เมื่อ query ข้อมูลออกมาก็จะได้เป็น object datetime ที่ time-zone-aware

Naive and aware datetime objects

อย่างที่รู้กันว่า `datetime.datetime` objects ของ Python นั้นมี attribute `tzinfo` ที่ใช้ในการเก็บข้อมูล time zone โดยถ้ามีการ set ค่า `tzinfo` ก็จะทำให้ datetime object นั้นเป็น time-zone-aware และถ้าไม่มีการ set ค่า `tzinfo` ก็จะเป็น time-zone-naive

Django จะมี `is_aware()` และ `is_naive()` ให้ใช้ในการเช็ค

```
from django.utils import timezone

now = timezone.now()
```

Source code ของ `timezone.now()` :

```
def now():
    """
    Returns an aware or naive datetime.datetime, depending on settings.USE_TZ.
    """
    if settings.USE_TZ:
        # timeit shows that datetime.now(tz=utc) is 24% slower
        return datetime.utcnow().replace(tzinfo=utc)
    else:
        return datetime.now()
```

เรามาลองดูตัวอย่างการใช้งานกัน

ก่อนอื่นไปแก้ไขไฟล์ `/week3/settings.py` ดังนี้

```
...
TIME_ZONE = "Asia/Bangkok"
```

```
USE_I18N = True
```

```
USE_TZ = True
```

```
...
```

เปิด shell ขึ้นมาแล้วลองทำตามด้านล่าง

```
>>> from zoneinfo import ZoneInfo
>>> from datetime import datetime
>>> from django.utils import timezone

>>> dt1 = datetime(2015, 12, 21, 12, 0, tzinfo=ZoneInfo(key='UTC'))
>>> print(dt1)
2015-12-21 12:00:00+00:00
>>> timezone.is_aware(dt1)
True

>>> dt1_local = timezone.localtime(dt1)
>>> print(dt1_local)
2015-12-21 19:00:00+07:00

>>> dt2 = datetime(2015, 5, 21, 12, 0)
>>> print(dt2)
2015-12-21 12:00:00
>>> timezone.is_aware(dt2)
False
>>> timezone.is_naive(dt2)
True

>>> dt2_aware = timezone.make_aware(dt2)
>>> print(dt2_aware)
2015-05-21 12:00:00+07:00
>>> timezone.is_aware(dt2_aware)
True
```